# IndERA-Drishti — Vision-Language Integrated 6-DOF Arm for Industry 4.0

Shivam Soni ([redacted]) & Kartikya Saini (24B4519)

**Abstract**

IndERA-Drishti is a six-degree-of-freedom (6-DOF) robotic arm system developed for Industry 4.0 applications, specifically targeting the end-of-life recycling phase of manufactured goods. Modern recycling demands highly adaptable systems capable of identifying and handling heterogeneous, unordered objects. The IndERA-Drishti system integrates three core subsystems: an Inverse Kinematics (IK) engine for precise 3D positioning using the Robotics Toolbox for Python, a PaliGemma vision-language model for zero-shot object classification, and a comprehensive digital twin for simulation and human-machine interface (HMI). A dual-Arduino hardware architecture controls the arm and a synchronized conveyor belt. This report details the theoretical foundations of the kinematics solver, the optimized vision inference pipeline, the distributed embedded hardware design, and the integration efforts toward a fully automated pick-and-place prototype

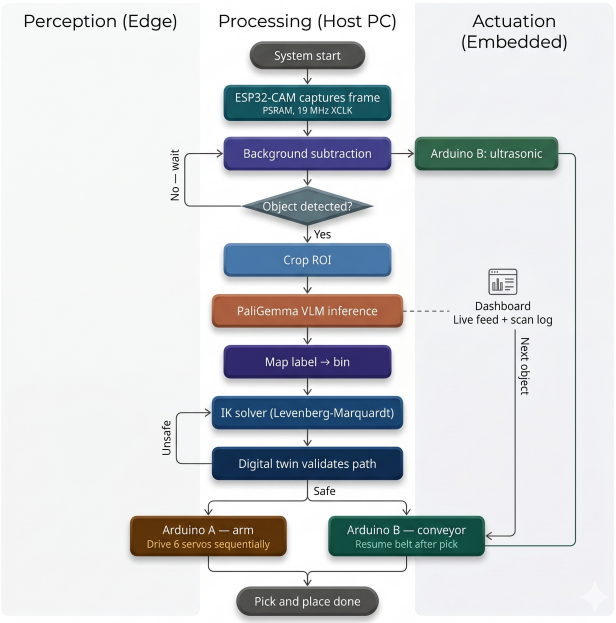



**Figure 1.** System Flowchart

## 1. Introduction and Motivation

The advent of Industry 4.0 has revolutionized manufacturing processes, yet the end-of-life (EoL) recycling phase remains a significant bottleneck. In a typical recycling facility, objects arrive in an unpredictable manner, varying vastly in size, orientation, and category. Traditional rule-based robotic systems struggle with this unstructured environment because they rely on predefined object models and rigid spatial programming.

IndERA-Drishti is proposed as a flexible, AI-driven solution to this challenge. The primary motivation is to build a system capable of autonomously observing an object on a moving conveyor, classifying it using advanced vision and natural language processing models, calculating the precise joint angles required to reach it, and physically sorting it into the correct inventory bin.

## 2. System Architecture

The IndERA-Drishti ecosystem is a distributed architecture that bridges high-level Python computing environments with low-level microcontrollers. The architecture ensures that computational heavy-lifting (AI inference and mathematical optimization) is isolated from real-time motor control.

The system functions across three computational tiers:

1. **Perception Tier (Edge):** An ESP32-CAM module captures real-time imagery of the conveyor belt.
2. **Processing Tier (Host PC):** A central Python pipeline orchestrates the logic. It receives image frames, performs background subtraction to localize the object, queries the PaliGemma model for classification, and computes joint angles using the IK solver.
3. **Actuation Tier (Embedded):** Two separate Arduino UNO microcontrollers handle physical outputs. Arduino A manages the robotic arm servos; Arduino B manages the conveyor's DC motor and reads the ultrasonic sensor.

## 3. Inverse Kinematics and URDF Modeling

For a robotic arm to successfully interact with its environment, it must map operational space coordinates $(x, y, z)$ into joint space coordinates. This process is known as Inverse Kinematics (IK).

### 3.1 Theoretical Foundation

Unlike Forward Kinematics, which uses predefined joint angles to compute the end-effector position, Inverse Kinematics is non-linear and often has multiple solutions. IndERA-Drishti utilizes the **Levenberg–Marquardt (LM) iterative method**. The iterative update rule is defined as:

$$\theta_{k+1} = \theta_k + (J^T J + \lambda I)^{-1} J^T e \tag{1}$$

where $J$ is the Jacobian matrix, $e$ is the positional error vector, $I$ is the identity matrix, and $\lambda$ is the damping factor that adjusts the step size for robust convergence near singularities.

### 3.2 Implementation and URDF Modeling

The core IK engine is built on top of the **Robotics Toolbox for Python**. To accurately represent our physical arm in the software space, we constructed a Universal Robotic Description Format (URDF) file.

By mapping joint intrinsics, axis orientations, and physical length limits within the URDF, the IK solver dynamically respects the boundaries of the physical hardware. Figure 2 illustrates the simulated kinematic chain operating within these designated limits.

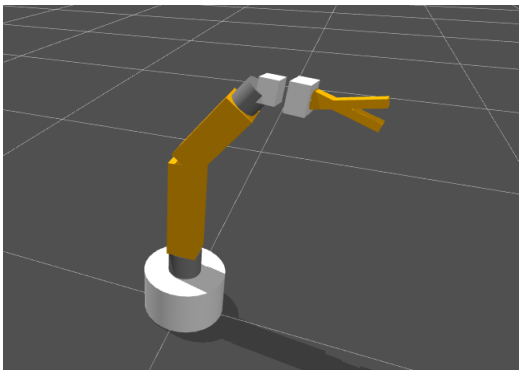


**Figure 2.** URDF model of the IndERA-Drishti arm rendered in the Swift 3D simulator, demonstrating a calculated extended pose to reach a coordinate boundary.

## 4. Digital Twin and Human-Machine Interface

A crucial component of modern Industry 4.0 systems is the Digital Twin, a virtual representation that mimics the state and behavior of the physical asset. Using the Swift 3D simulator integrated with the Robotics Toolbox, we integrated a fully responsive digital replica of IndERA-Drishti with all parts of our pipeline.

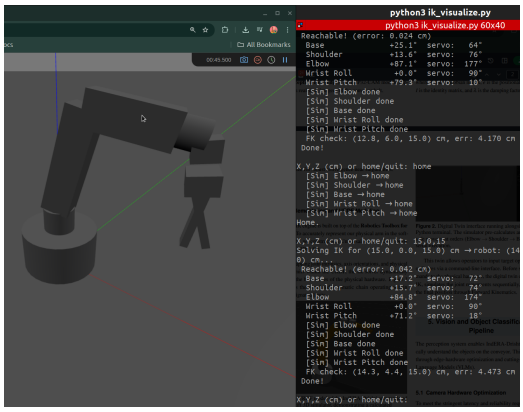



**Figure 3.** Digital Twin interface running alongside the Python terminal. The simulator pre-calculates and animates safe movement orders

This twin allows operators to input target operational coordinates via a command-line interface. Before sending commands to the physical hardware, the digital twin computes the IK, simulates the joint movements sequentially, and verifies the final position through Forward Kinematics.

## 5. Vision and Object Classification Pipeline

The perception system enables IndERA-Drishti to dynamically understand the objects on the conveyor. This is achieved through edge-hardware optimization and cutting-edge Vision-Language Models (VLMs).

### 5.1 Camera Hardware Optimization

To meet the stringent latency and reliability requirements of the pipeline, lower-level optimizations were applied to the ESP32-CAM firmware:

- **PSRAM Allocation:** Framebuffers were explicitly forced to utilize the external Pseudo-SRAM (PSRAM) to prevent memory fragmentation.
- **Dual Processing Disabled:** Dual-core processing was disabled to ensure deterministic behavior during image acquisition.
- **Clock Underclocking:** The XCLK was iteratively underclocked to 19 MHz. While slightly reducing framerate, this drastically improved signal integrity and reduced frame tearing on the moving conveyor.

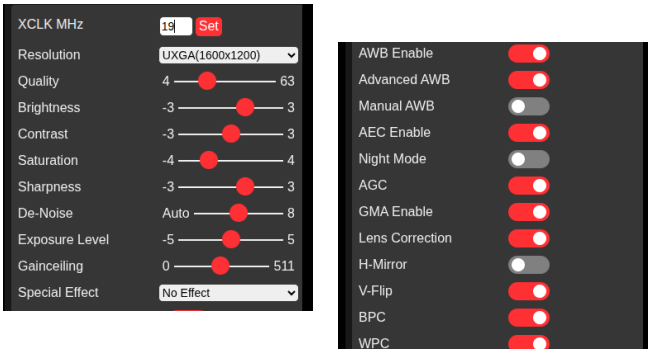
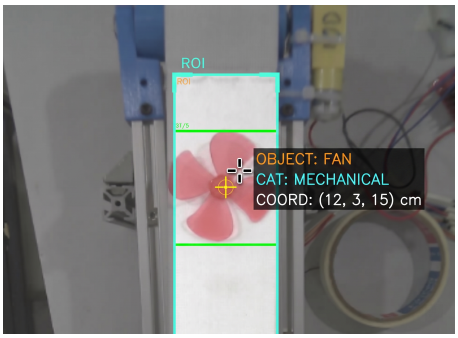


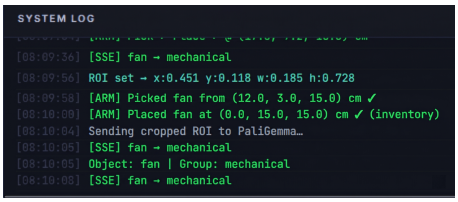

**Figure 4.** ESP32 OV3660 camera parameter configuration, showcasing the custom XCLK adjustments applied to stabilize the video feed

### 5.2 Detection and PaliGemma Classification

Once a frame is captured, a background subtraction algorithm isolates the object from the conveyor background to determine its pixel centroid and dimensions (Figure 5).




**(a)** ROI extraction and coordinate mapping.




**(b)** System event log tracing the operation.

**Figure 5.** Vision pipeline capturing a fan, calculating base frame coordinates, and logging the pick-and-place execution sequence.

The cropped Region of Interest (ROI) is passed to **PaliGemma**, an open-source vision-language model. Instead of relying on a rigidly trained classifier, we prompt PaliGemma in natural language to identify the object. During testing, PaliGemma successfully categorized items such as plastic fan, office tape, industrial lab Lego wheels, metal screwdrivers, and electronic components. The output results are displayed on the custom built web dashboard (Figures 6 and 7).

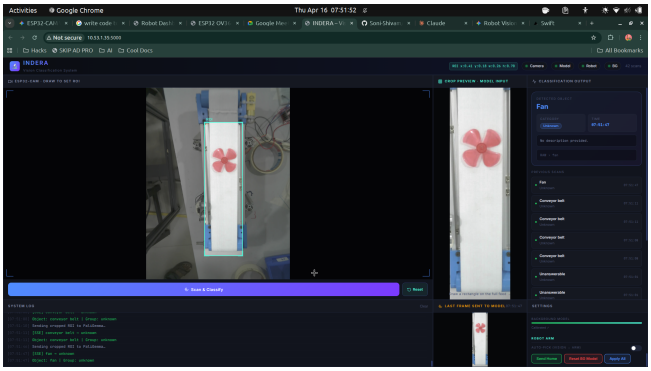


**Figure 6.** IndERA-Drishti Dashboard streaming the live conveyor feed and displaying live PaliGemma inference results (e.g., detecting a fan).

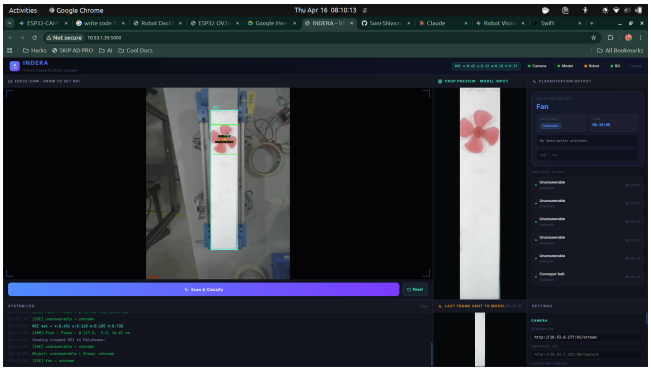


**Figure 7.** Dashboard view highlighting the bounding box overlay, extracted model input preview, and chronological scan history.

## 6. Industry 4.0 Integration

The IndERA-Drishti system embodies the core tenets of Industry 4.0 by transitioning a traditional mechanical manipulator into a decentralized cyber-physical system. Our architecture relies on several ”smart” integration elements to handle data and execute autonomous decisions:

- **Autonomous Cognitive Decisions:** Traditional robotic sorters rely on rigid spatial programming. By integrating the PaliGemma VLM, IndERA-Drishti autonomously reasons about entirely unseen objects. It processes unstructured visual data and makes zero-shot semantic sorting decisions (e.g., dynamically categorizing a stray PCB as ”Electronics”) without human intervention.
- **Distributed Edge-to-Host Data Pipeline:** The system utilizes an IoT-inspired data flow. The ESP32-CAM and ultrasonic sensors act as edge nodes capturing real-time environmental triggers. This lightweight data is passed to the central Python host, which handles the heavy-compute AI inference and non-linear IK mathematics,

before transmitting distilled, sequential execution arrays to the embedded actuation layer.

- **Cyber-Physical Synchronization:** The Swift 3D Digital Twin acts as a live virtual mirror. By pre-calculating and validating kinematic trajectories in the digital space before physical execution, the system ensures data-driven, collision-free operations, significantly reducing the risk of hardware failure on the factory floor.

## 7. Hardware and Embedded Architecture

The physical execution relies on a rigidly separated dual-microcontroller architecture to prevent blocking functions from interrupting the robotic arm's smooth movement profiles. Figure 8 illustrates the complete hardware topology.

### 7.1 Arduino A: Arm Kinematics Controller

Arduino A is entirely dedicated to driving the six servo motors of the robotic arm. It is stripped of all sensor responsibilities.

The control pipeline relies on a Python script running on the host machine, which communicates with Arduino A via the `PySerial` library. Once the IK solver determines the correct joint angles, Python formats these angles into a structured array and transmits them over a serial connection. Arduino A runs a streamlined embedded C++ program that listens to the serial port, accepts the angle array, and processes them sequentially, avoiding peak stall currents and preventing mechanical self-collision.

### 7.2 Arduino B: Conveyor and Sensor Controller

Arduino B functions as the factory floor manager. It is isolated from the complex kinematics and interfaces solely with:

- An **HC-SR04** Ultrasonic sensor to detect approaching objects.
- A **DC motor** driven by an L298N motor driver module, controlling the conveyor belt.
- A **potentiometer** to manually trim and adjust conveyor speed dynamically.

Arduino B operates autonomously: when the ultrasonic sensor detects an object within a predefined threshold, it halts the L298N driver. The paused state allows the camera system to capture an unblurred image, calculate coordinates, and trigger the IK sequence.

## 8. Challenges and Learnings

Developing the IndERA-Drishti pipeline presented several significant technical hurdles across hardware, software, and physical modeling. Overcoming these challenges required moving beyond theoretical design and applying practical debugging strategies.

### 8.1 Coordinate Frame Mismatches and Transformations

One of the most complex logical hurdles was resolving the spatial discrepancies between three distinct coordinate frames: the Swift 3D simulator, the native IK solver, and the physical IndERA arm. For instance, a target coordinate of $(12, 12, 12)$ calculated by the IK solver mapped to approximately $(14.8, 8, 12)$ in the physical real-world frame. To rectify this, we had to derive and apply a custom matrix transformation to properly translate and align the base frames before sending the final angular payloads to the servos.

### 8.2 Kinematic Precision and URDF Modeling

Constructing the URDF model from scratch was a novel challenge for our team. Capturing the precise physical dimensions, joint offsets, and mass distributions of the IndERA arm was difficult but critical, as any millimeter-level discrepancy in the URDF compounded into significant positional errors at the end-effector. Furthermore, we observed that the physical execution of the IK solutions lacked perfect precision in the real world. Mechanical backlash in the servos and structural flexing of the links introduced slight deviations from the theoretical calculations.

### 8.3 Hardware Failures and Power Limits

During the iterative testing phase, we experienced significant hardware attrition, blowing out a total of seven servo motors. This was primarily caused by aggressive movement profiles and stalling loads when the IK solver occasionally commanded the arm into mechanically stressful configurations before joint limits were strictly enforced in the software. This costly learning curve highlighted the critical importance of power rail isolation and sequential, delayed motor actuation.

### 8.4 Perception Latency and Pipeline Integration

Integrating the distributed pipeline (Edge Camera $\rightarrow$ Python/-PaliGemma Host $\rightarrow$ Dual-Arduinos) introduced substantial synchronization issues. The ESP32-CAM suffered from inherent network streaming latency. When combined with the inference time of the Vision-Language Model and the IK computation, the total pipeline delay meant the physical object on the conveyor often drifted out of the target pick zone before

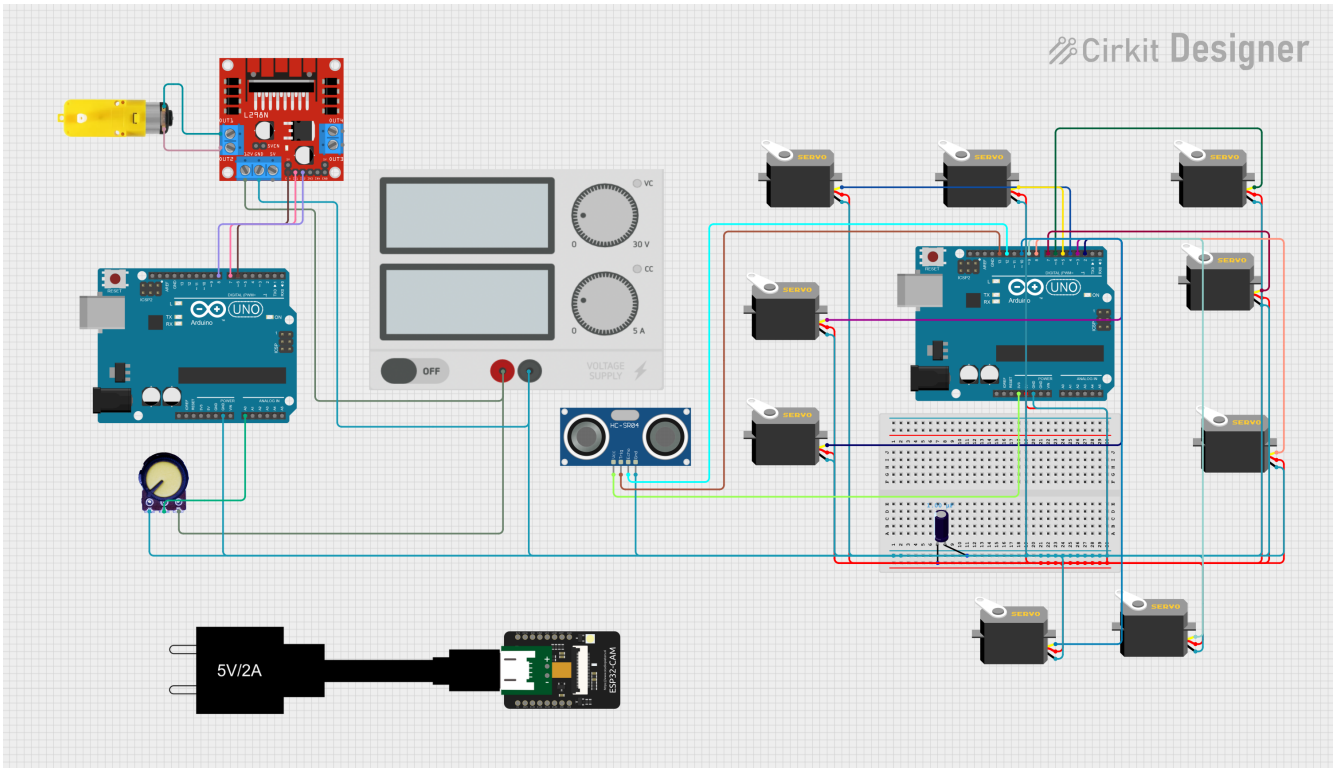



**Figure 8.** Comprehensive circuit schematic detailing the isolated Dual-Arduino logic layout, ESP32-CAM power delivery, L298N motor driver, and the 6-DOF servo array.



**Figure 9.** Our team and IndERA-Drishti (Hehe..)

the arm could reach it. This exact issue caused too much of our time to be utilised hence finally we could only build the individual parts but never the complete pipeline.

## 9. Conclusion and Future Work

The IndERA-Drishti project successfully demonstrates a highly modular, intelligent approach to robotic automation in Industry 4.0 contexts. By decoupling the architecture into a high-level AI/Kinematics brain (Python, PaliGemma, Robotics Toolbox) and a low-level actuation nervous system (Dual Arduinos), the system achieves both high computational intelligence and reliable physical execution.

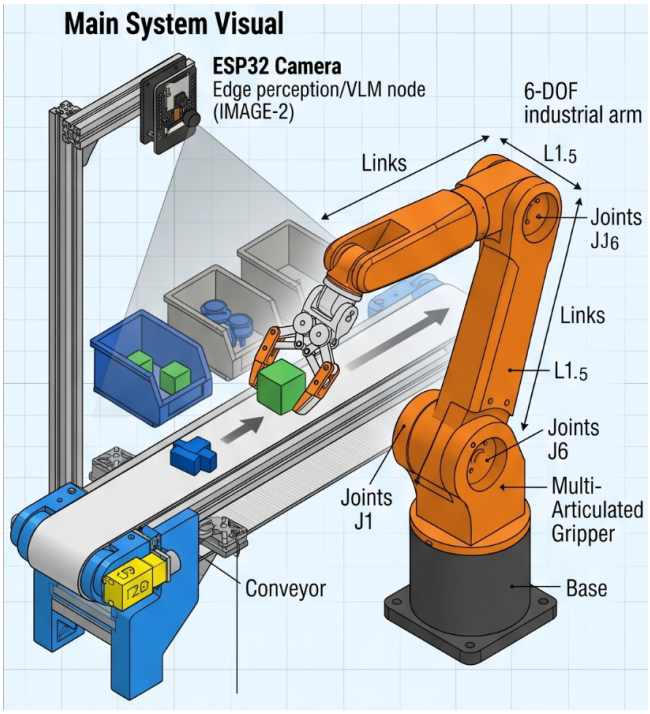



**Figure 10.** Poster Visual of IndERA-Drishti

While the individual subsystems demonstrated high efficacy, achieving a fully fluid, uninterrupted integration proved challenging within the time constraints. The primary bottleneck was bridging the dynamic coordinate transform between the 2D bounding box centroids and reliable 3D world space positioning.

Future iterations will focus on implementing an automated calibration system to dynamically update camera intrinsics, and completing the seamless serial handshake between Arduino A and Arduino B to eliminate any manual intervention during the continuous sorting cycle.

## 10. Code Files

The complete software stack and hardware schematics developed for the IndERA-Drishti project are attached.

The primary repository includes the Python vision-kinematics pipeline, the custom URDF configuration files, the Next.js/FastAPI web dashboard source code, and the distributed embedded C++ Arduino sketches:

- **IndERA-Drishti Main Repository:**
  https://github.com/Soni-Shivam/IndERA-Drishti